

۱۳۰۷

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشکده مهندسی کامپیوتر

پروژه مقطع کارشناسی مهندسی کامپیوتر درس سیستم های عامل

پیاده سازی سیستم فایل

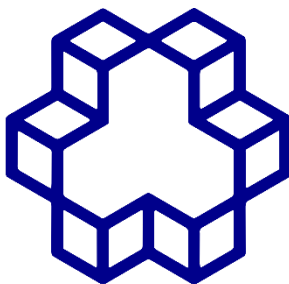
محمد حسین شعبانی راد

استاد راهنما:

دکتر یعقوبی

نیمسال اول تحصیلی ۱۴۰۳-۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه صنعتی خواجه نصیرالدین طوسی

اظهاری نامه دانشجوی

اینجانب **محمد حسین شعبانی** راد دانشجوی مقطع کارشناسی رشته مهندسی کامپیوتر گواهی می‌نمایم که مطالب ارائه شده در این پروژه با عنوان:

پیاده سازی سیستم فایل

با راهنمایی استاد محترم **دکتر یعقوبی** توسط شخص اینجانب انجام شده است. صحت و اصالت مطالب نوشته شده در این پروژه تأیید می‌شود و در تدوین متن پروژه قالب مصوب دانشگاه را به طور کامل رعایت کرده‌ام.

امضاء دانشجو:

تاریخ:

حق طبع، نشر و مالکیت نتایج

- 1- حق چاپ و تکثیر این پروژه متعلق به نویسنده و استاد راهنمای آن است. هرگونه تصویربرداری از کل یا بخشی از پروژه تنها با موافقت نویسنده یا استاد راهنما یا کتابخانه دانشکده‌های مهندسی برق و کامپیوتر دانشگاه صنعتی خواجه نصیرالدین طوسی مجاز است.
- 2- کلیه حقوق معنوی این اثر متعلق به دانشگاه صنعتی خواجه نصیرالدین طوسی است و بدون اجازه کتبی دانشگاه قابل واگذاری به شخص ثالث نیست.
- 3- استفاده از اطلاعات و نتایج موجود پروژه بدون ذکر مرجع مجاز نیست.

چکیده

دستگاه های ذخیره سازی روز به روز پیشرفته تر شده و نحوه ذخیره سازی داده ها در آنها به شکل های متفاوتی انجام میشود . یکی از مسائل چالش برانگیز در رابطه این است که یک دستگاه ذخیره سازی با سیستم فایل تعیین شده اش در انواع سیستم های عامل قابل ذخیره و بازیابی اطلاعات باشد. سیستم فایل یکی از بخش های مهم ارتباطی هسته سیستم عامل با حافظه است و تعامل با انواع حافظه ها به وسیله آن انجام خواهد شد . لذا اهمیت چگونگی پیاده سازی سیستم فایل ها حائز اهمیت است .

در این پروژه ابتدا معماری سیستم فایل متناسب با کاربرد و نحوه عملکرد آن تعیین شده مزیت ها معایب آن و راه حل های احتمالی گفته خواهد شد سپس به پیاده سازی آن برای اجرا در ماشین اجرایی جاوا پرداخته خواهد شد . برای استفاده از این سیستم فایل دستورات عملکردی اصلی نیز پیاده سازی خواهد شد در بخش های مربوطه اشاره به نحوه عملکرد آنها خواهد شد .

کلید واژه: سیستم های عامل ، سیستم فایل ها ، جاوا

فصل 1- مقدمه

1-1- پیشگفتار

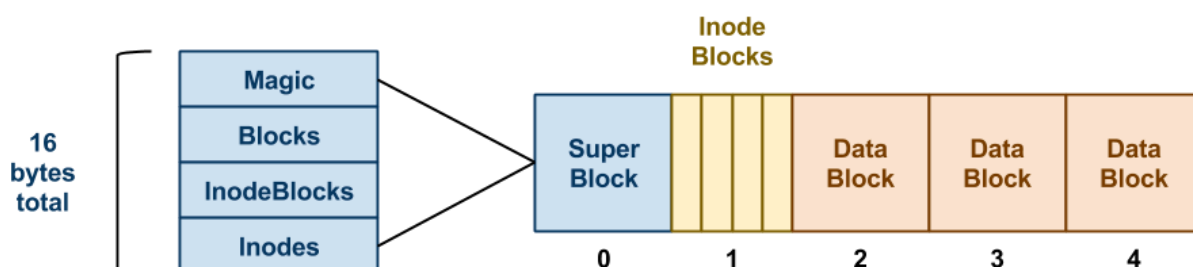
فایل مجموعه ای از اطلاعات رابطه ای است. سیستم فایل بر روی حافظه دومی قرار دارد که این امکان را به داده ها می دهد که برای ذخیره، مکان یابی و بازیابی، دسترسی کارآمد و راحت به دیسک چگونه تعامل کنند. پیاده سازی سیستم فایل در یک سیستم عامل به نحوه مدیریت فایل سیستم ذخیره سازی و بازیابی داده ها در یک دستگاه ذخیره سازی فیزیکی مانند هارد دیسک، درایو حالت جامد یا درایو فلش اشاره دارد. اجرای سیستم فایل یک جنبه حیاتی از یک سیستم عامل است زیرا به طور مستقیم بر عملکرد، قابلیت اطمینان و امنیت سیستم تاثیر می گذارد. سیستم عامل های مختلف بر اساس نیازهای خاص سیستم و موارد استفاده مورد نظر از پیاده سازی های سیستم فایل متفاوتی استفاده می کنند. برخی از سیستم های فایل رایج مورد استفاده در سیستم عامل ها عبارتند از NTFS و FAT در ویندوز و ext4 و XFS در لینوکس. (2)

1-2- هدف از پروژه

تجربه پیاده سازی ساده ای از سیستم فایل ها در ماشین اجرایی جاوا نحوه پیاده سازی سیستم فایل های اولیه و نحوه عملکرد دستورات فایل ای در آنها

فصل 2- انتخاب ساختار سیستم فایل

با بررسی های انجام شده و مطالعه انواع سیستم فایل ها در سیستم های متفاوت ساختار کلی بدست آمده از سیستم فایل ها که در شکل شماره یک فایل مشاهده است نتیجه شد . (3) (1)

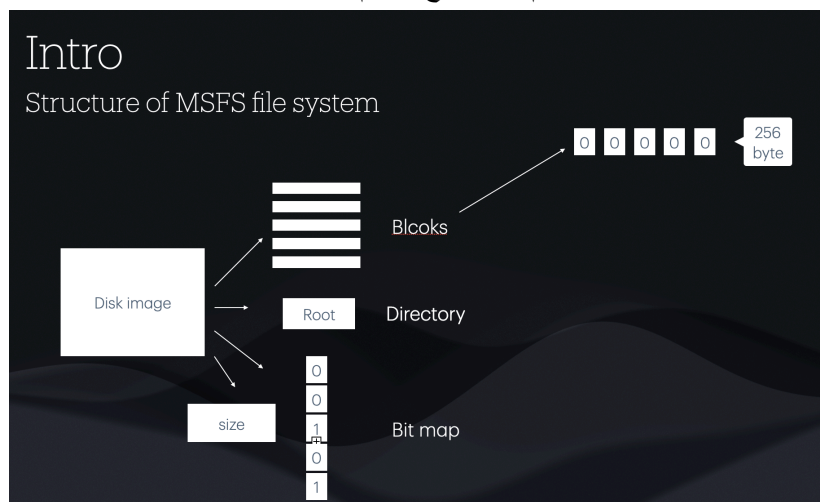


شکل ۱ - ساختار کلی هر فایل در سیستم فایل

بطور کلی سوپر بلوک تعیین کننده و شناسه آن است که سیستم فایل ما از چه نوع است و اطلاعات اصلی آن در آن است . در بخش Inode ما اطلاعات هر بلوک از داده شامل آزاد بودن یا نبودن ارتباط با بقیه بلوک ها سطح دسترسی و ... را ذخیره خواهیم کرد.

در نهایت این بررسی ها یکی از عوامل تعیین کننده بهینه بودن سیستم فایل انتخاب سائز هر بلوک داده است . این موضوع از آن جهت اهمیت دارد که فرض کنید سائز بلوک اندازه یک دیسک انتخاب شود با این انتخاب شما تنها مجاز به انتقال یک فایل خواهیم بود . از طرفی اگر به اندازه زیاد کوچک باشد پیچیدگی ارتباط بین بلوک ها افزایش خواهد یافت .

حال ساختار پیاده شده در پروژه را در شکل زیر می بینیم که توضیح خواهیم داد

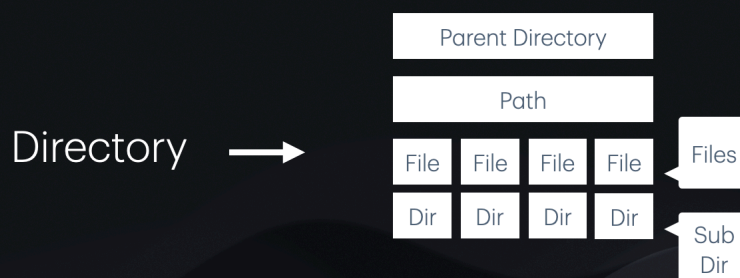


شکل ۲ - ساختار کلی سیستم فایل MSFS

از این جا به بعد نام سیستم فایل پروژه به اختصار **MSFS** که از اسم خودم انتخاب شده گفته خواهد شد. هر دیسک از این سیستم فایل شامل یک بیت مپ به اندازه تعداد بلوک ها خواهد بود که نشان دهنده خالی بودن یا نبودن آنها است. با توجه به استفاده از مفهوم شی گرایی جاوا در پروژه یک شی از نوع دایرکتوری نیز داریم که نشان دهنده دایرکتوری مادر و دایرکتوری های فرزند آن است که در ادامه با شکل توضیح داده خواهد شد. با توجه به اهمیت سایز هر بلوک در این سیستم فایل سایز هر بلوک 256 بایت انتخاب شده یعنی هر ۴ واحد از این بلوک یک کیلوبایت خواهد شد. و در نهایت خود بلوک ها بقیه دیسک را خواهند گرفت.

Intro

Structure of MSFS file system

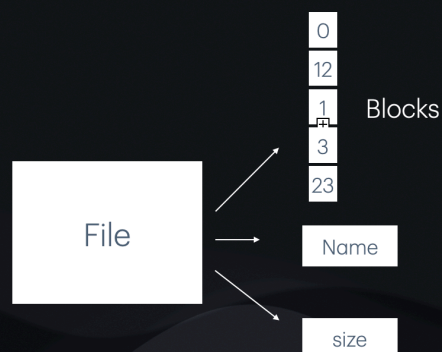


شکل ۳ - ساختار دایرکتوری های MSFS

هر دایرکتوری شامل یک **path** یا مسیر است که برای دایرکتوری مادر / است .
هر دایرکتوری اشاره به دایرکتوری مادر خود نیز باید داشته باشد تا قابل بازگشت باشد .
در نهایت لیست فایل و دایرکتوری های این دایرکتوری نیز باید باشد تا دسترسی به هرکدام از آنها محدود شود .

Intro

Structure of MSFS file system



شکل ۴ - ساختار فایل های MSFS

هر فایل باید بداند که داده هایش در کدام بلوک های دیسک قرار دارد که شماره بلوک ها داده میشود .
نام فایل نیز وجود دارد .
همچنین مقدار آن جهت مدیریت بلوک هایش انتخاب خواهد شد .

فصل 3- اثر عملیات ها بر فایل ها

دستوراتی که در این پروژه پیاده سازی شدند عبارت اند از :

دستورات عملیات بر روی دایرکتوری :

۱- mkdir

۲- ls

۳- cd

۴- rm

به ترتیب برای ایجاد یک دایرکتوری در دایرکتوری حاضر ، لیست دایرکتوری های فرزند ، تغییر دایرکتوری حاضر و حذف یک دایرکتوری استفاده خواهند شد .

دستورات عملیات بر روی فایل :

۱- touch

۲- cat

۳- rmf

به ترتیب برای ایجاد یک فایل یا اصلاح آن در دایرکتوری حاضر ، نشان دادن محتوای فایل و حذف یک فایل استفاده خواهند شد .

اثر هر عملیات در جدول قابل مشاهده است که مشابه با پیاده سازی (4) Three Easy Pieces است :

عملیات	Blocks	Directory	Bitmap
Mkdir	-	+ ایجاد دایرکتوری جدید	-
Ls	-	خواندن فایل و دایرکتوری های فرزند	-
Cd	-	-	-
Rm	به اثر فایل های آن	+	+ به سبب حذف فایل های مربوط به آن
Touch	در اثر ایجاد محتوا	+ اضافه شدن فایل	+ در اثر تغییر فایل
Cat	خواندن از آن به سبب مپ	-	تعیین خواندن کدام بلوک
RMF	آزاد سازی	حذف از دایرکتوری مربوطه	تغییر بلوک های وابسته از ۱ به ۰

جدول ۱ - اثر عملیات در ساختار سیستم فایل

فصل 4- پیاده سازی سیستم فایل

1. دیسک (Disk)

در شروع شی دیسک که ماهیت کلی تمام فایل و فولدر ها قرار است در آن باشند پیاده سازی میشود :

```
class FileSystem implements Serializable {

    //private static final int TOTAL_BLOCKS = 4096 * 5; // File system size
    (1MB with 256-byte blocks)

    final int TOTAL_BLOCKS;

    Block[] blocks;

    BitSet bitMap;

    Directory rootDirectory = new Directory("/", null);

    Directory currentDirectory = rootDirectory;

    public FileSystem(int size) {

        TOTAL_BLOCKS = size / 256;

        blocks = new Block[TOTAL_BLOCKS];

        bitMap = new BitSet(TOTAL_BLOCKS);

        for (int i = 0; i < TOTAL_BLOCKS; i++) {

            blocks[i] = new Block();

        }

    }

}
```

ساختار هر بلوک به این شکل است که آرایه ای ۲۵۶ بایتی خواهد بود . با توجه به سایز دیسک لیستی از این بلوک ها به تعداد مشخص بدست خواهد آمد که در موقع ایجاد باید ساخته شوند .

2. دایرکتوری (Directory)

به اختصار پیاده سازی متدهای شی گذاشته نشده است .

```
class Directory implements Serializable {
    private String name;
    public String path = "/";
    private Map<String, File> files = new HashMap<>();
    private Map<String, Directory> subDirectories = new HashMap<>();
    private Directory parent;

    public Directory(String name, Directory parent) {
        this.name = name;
        this.parent = parent;
        if (parent != null) {
            path = parent.path + name + "/";
        }
    }

    public String getName()
    public Directory getParent()
    public Map<String, File> getFiles()
    public Map<String, Directory> getSubDirectories()
    public boolean addFile(File file)
    public File removeFile(String fileName)
    public boolean addDirectory(Directory dir)
}
```

کد ۲- شی دایرکتوری

3. فایل (File)

```
class File implements Serializable {
    private String fileName;
    private List<Integer> blockPointers = new ArrayList<>();
    private int size;

    public File(String fileName) {
        this.fileName = fileName;
        this.size = 0;
    }

    public void addBlock(int blockIndex)
    public List<Integer> getBlockPointers()
    public String getFileName()
    public int getSize()
    public void setSize(int size)
}
```

کد ۳- شی فایل

پیاده سازی تابع های کمکی از جمله مدیریت ورودی های کاربر و هندل کردن فایل ها نیز در کد منبع در دسترس است . (5)

در نهایت با توجه به اینکه پروژه با زبان جاوا نوشته شده برای سادگی اجرا و شبیه به واقعیت یک بش (Bash) برای آن نوشته شده :

```
#!/bin/bash

JAR_PATH="./MyApp.jar"
java -jar ./MyApp.jar "$@"
```

کد ۴- فایل بش (Bash)

فصل 5- گزارش اجرا

حال نحوه استفاده از پروژه گفته میشود و :

ساخت یک دیسک جدید :

Create [disk_name] [size] [unit]

این دستور باعث اجرا شدن کد زیر میشود :

```
else if (args[0].equals("create") && v == 4) {  
  
    fs = Helper.loadFileSystem(args[1]);  
  
    if (fs != null) {  
  
        System.out.println(args[1] + " disk is already exist and now open!");  
  
    } else {  
  
        fs = new FileSystem(Helper.byteSize(args[2], args[3]));  
  
        Helper.saveFileSystem(args[1], fs);  
  
        System.out.println(args[1] + " disk created!");  
  
    }  
  
}
```

کد ۵ - دستور ایجاد دیسک

اگر دیسکی از قبل داریم میتوانیم از دستور زیر برای وارد شدن به آن استفاده کنیم :

mount [disk_name]

این دستور باعث اجرا شدن کد زیر میشود :

```
if (args[0].equals("mount") && v == 2) {  
  
    fs = Helper.loadFileSystem(args[1]);  
  
    if (fs == null) {  
  
        return;  
  
    }  
  
}
```

کد ۶ - دستور باز کردن دیسک

نتیجه اجرا این بخش به شکل زیر خواهد بود :

```
mounting disk image ....  
mmd2 disk is already exist and now open!  
MSFS>>mmd2.disk/
```

شکل ۵ - نتیجه دستور باز کردن دیسک

حال بررسی می کنیم موقع ایجاد یک فایل چه اتفاقی می افتد :

- اگر فایل از قبل وجود نداشته باشد متود زیر صدا زده میشود :

```
public static void createFile(FileSystem fs, String fileName)
```

در غیر این صورت :

```
public static void writeFile(FileSystem fs, String fileName, byte[]  
data)
```

ما در این حالت اگر یک شی فایل به اسم fs داشته باشیم زیر روال زیر برای آن صدا زده میشود :

- پیدا کردن اولین بلوک خالی برای آن
- تخصیص آن
- نوشتن داده در بلوک ها
- آپدیت سائز فایل


```
fs.bitMap.nextClearBit(0);  
  
fs.bitMap.set(freeBlock);  
  
file.addBlock(freeBlock);  
  
fs.blocks[freeBlock].writeData(Arrays.copyOfRange(data, dataOffset,  
dataOffset + writeSize), 0);  
  
file.setSize(file.getSize() + writeSize);
```

فصل 6- جمع‌بندی

با توجه به اینکه سادگی هدف اصلی این پروژه و عدم دخیل کردن پیچیدگی های لینک لیست شدن برای Inode ها بوده همچنین رفع مشکل پیچیدگی ذخیره فایل با استفاده از Serializable جاوا پروژه عملکرد بهتری داشت .
با این حال ما از معایب آن نیز باید بگوییم .
با توجه به استفاده از قابلیت ذخیره شی ها در جاوا خود این زبان نیز یک سوپر بلاک دیگری برای آنها خواهد گذاشت که این موضوع موجب شده اندازه هر دیسک شاید از یک جا به بعد از حد انتظار بیشتر شود
به عنوان مثال این موضوع برای یک دیسک ۵۰۰ مگابایتی حدود ۵۰ مگابایت هزینه گذاشته است .

در نهایت با تشکر از توجه شما نسبت به این پروژه داکيومنت انگلیسی استفاده از پروژه و اسلاید ارائه آن در ریپوزیتوری گیت به همراه کد در دسترس است

فصل 7- فهرست مراجع

1. Bui, Peter. "[Simple File System operating system](https://www3.nd.edu/~pbui/teaching/cse.30341/fa17/)." *Operating System Principles*, 2017, <https://www3.nd.edu/~pbui/teaching/cse.30341/fa17/>. Accessed 1 1 2017.
2. Jain, Sandeep. "File System Implementation in Operating System." *GeeksforGeeks*, 24 July 2024, <https://www.geeksforgeeks.org/file-system-implementation-in-operating-system/>. Accessed 5 January 2025.
3. Saraç, Mustafa. "File-System." *Spring 2017 COMP 304 Operating Systems Course Project*, Koç University, 2017, <https://github.com/saracmustafa/File-System/tree/master>. Accessed 1 1 2017.
4. Agrawal, Nitin, and William J. Bolosky. "File System Implementation." *cs.wisc.edu*, 2023, <https://pages.cs.wisc.edu/~remzi/OSTEP/file-implementation.pdf>. Accessed 5 January 2025.
5. Shabaniraad, Mohammadhosein. "MSFS File System." *MSFS File System github*, 2024, <https://github.com/mohamadhoseinraad/MSFS>. Accessed 1 10 2024.